

Payment Collection AI Agent

Design Document

Author: Himanish Goel | Role: Agent Engineer | Stack: Python 3.10+, Ollama (llama3.1:8b), FastAPI, LangChain-Ollama

1. Architecture Overview

The agent is structured as a deterministic checklist state machine with the LLM used only at the edges — extraction and response generation. All verification logic, card validation, and business rules are pure deterministic Python with zero LLM in the critical path.

Stage Lifecycle

```
INIT → AWAIT_ACCOUNT_ID → AWAIT_NAME → AWAIT_SECONDARY
      → AWAIT_AMOUNT → AWAIT_CARD → [CONFIRM_PAYMENT] → DONE
                                                    → LOCKED_OUT
                                                    → ERROR
```

Core Modules

Module	Responsibility	LLM?
agent.py	Turn routing, state transitions, inverted gateway	No
extractor.py	NLP extraction: account ID, name, DOB, card, amount	Yes (fallback only)
verifier.py	Identity verification: name + secondary factor	No
responder.py	Natural language response generation (persona: Niki)	Yes
tools.py	External API wrappers with retry/backoff	No
state.py	Typed conversation state + Stage enum	No
api.py	FastAPI server with session management	No

2. Key Design Decisions

2.1 Inverted Gateway Pattern

The most important architectural decision. Instead of always routing through the LLM, every turn first runs `_sweep_checklist_inputs()` to opportunistically extract structured data. If any field changed, the LLM router (`classify_intent`) is skipped entirely. Only genuinely ambiguous turns (no data extracted) are routed through the SLM for intent classification.

Why: Reduces latency on clean inputs (typically 3–5 LLM calls per turn instead of 6–7), eliminates false negatives from the router, and makes the happy path faster and more deterministic.

2.2 Two-Layer Extraction: Regex First, LLM Fallback

Every extractor follows the same pattern:

- Fast deterministic regex (handles 80%+ of inputs)
- LLM JSON fallback for messy/colloquial input
- 1-to-1 Python substring validation — every LLM-extracted value must be a literal substring of the user's actual text

The third step is critical. It prevents the LLM from hallucinating values not present in the message.

2.3 Strict Case-Sensitive Name Verification

The verifier performs exact string match on the name, but adds a second check: even if the normalized `provided_name` matches, the verifier also confirms that the exact casing appears somewhere in the raw `state.history` user turns. This prevents LLM auto-capitalization from silently bypassing the case-sensitive rule — a subtle but real attack vector.

2.4 History Redaction (Blind LLM Pattern)

Before any turn's input is saved to `state.history`, `redact_user_input()` replaces all sensitive values (card number, DOB, Aadhaar, pincode, CVV) with `[FIELD: True]` flags. The LLM generating responses therefore never sees raw sensitive data in the history.

2.5 CVV Guard

When the card number is already collected and the user sends a standalone 3–4 digit message, the sweep explicitly suppresses `extract_amount()` and treats the input as a CVV. Without this guard, '123' would ambiguously match both the CVV and a ₹123 payment amount.

2.6 Cardholder Name Defaulting

Per spec, `cardholder_name` is accepted as-is and not validated against the account holder. If the user never provides one, the agent silently defaults it to the verified `account_name` before the API call. This avoids prompting the user for a field they may not know is required.

2.7 Gated Account Swapping

Before verification is complete, a user may correct their account ID. The agent detects a new ACC ID and wipes all stale state (account data, name, secondary factors, amount) to prevent cross-account contamination. This is locked down post-verification: `state.verified` must be `False` for a swap to be accepted.

2.8 Payment Guarantee Appends

After generating the success message via LLM, the agent performs two deterministic post-checks:

- If `'txn_'` or `'transaction'` does not appear → append the transaction ID
- If the paid amount string does not appear → append `'Amount Paid: ₹X'`

This guarantees 100% pass rate on evaluation checks that scan for the transaction ID or amount in the final message, regardless of how the LLM rephrases the success text.

3. Verification Logic

Implemented entirely in verifier.py — no LLM involved.

```
# Pass condition:
name_ok = provided_name == account_data['full_name'] # exact, case-sensitive
AND
(provided_dob == account_data['dob']
 OR provided_aadhaar == account_data['aadhaar_last4']
 OR provided_pincode == account_data['pincode'])
```

Retry limit: 3 attempts. Each failed attempt clears provided_name, provided_dob, provided_aadhaar, and provided_pincode so the user must provide fresh credentials on retry. After 3 failures, the session transitions to LOCKED_OUT and no further state changes are accepted.

Anti-exposure: account_data is stored in state but never serialized into any response or history entry. The balance and account_name properties are the only fields ever surfaced, and only after successful verification.

4. Card Validation & Payment Flow

Card Validation (deterministic, pre-API)

- Card number: 15–16 digits, Luhn check passes
- Expiry: month 1–12, year in range, not expired (checked against datetime.today())
- CVV: 3–4 digits (string, not int — preserves leading zeros like '007')
- Cardholder name: present (defaulted to account name if absent)

Invalid fields are reported specifically and only the offending field is cleared for re-entry — the user is not asked to re-enter the whole card.

Payment Error Classification

Error Code	Retryable	Action
invalid_card	Yes	Clear card number
invalid_cvv	Yes	Clear CVV
invalid_expiry	Yes	Clear expiry month + year
invalid_amount	Yes	Clear amount, return to AWAIT_AMOUNT
network_error	No (retry in tools.py)	Hardcoded reassurance message
insufficient_balance	No	Close gracefully
Any other	No	Transition to ERROR

Network Resilience (tools.py)

All API calls use exponential backoff with jitter: up to 3 retries on 502/503/504 and connection/timeout errors. The backoff formula is $\text{base} * 2^{(\text{attempt}-1)} + \text{uniform}(0.1, 0.5)$ seconds. Terminal exceptions are not retried.

5. Tradeoffs Accepted

LLM for Extraction vs. Pure Regex

Aspect	Detail
Chosen	Hybrid — regex fast-path + LLM fallback + substring validation
Tradeoff	LLM fallback adds ~800ms latency per extraction turn but handles colloquial inputs (word numbers, date formats) that pure regex would miss
Alternative Considered	Pure rule-based NLP (spaCy/Duckling) — rejected due to higher engineering cost, and failure on informal Indian English inputs. It has the ability to perform better than regex but needs more development time.

Local Ollama vs. Cloud LLM

Aspect	Detail
Chosen	Local Ollama (llama3.1:8b default)
Tradeoff	Lower latency, no API costs, full data privacy — but limited to ~8B parameter models (~4.5GB VRAM) on my RTX 4070
Alternative Considered	OpenAI/Anthropic APIs — viable for production but adds external dependency, cost, and potential data exposure

In-Memory Session Store vs. Persistent State

Aspect	Detail
Chosen	In-memory dict with 30-minute TTL
Tradeoff	Simple and fast, but sessions are lost on server restart — a shared store (Redis) would be required for multi-process deployments.

History Redaction vs. No History

Aspect	Detail
Chosen	Store sanitized history (sensitive fields replaced with [FIELD: True])
Tradeoff	The LLM has conversational context for warm, coherent responses, but never sees raw PII. No-history alternative would produce more robotic, context-blind responses

6. What I Would Improve With More Time

Short-Term (1–2 weeks)

- Streaming responses: Return LLM output tokens as a server-sent event stream so the UI shows the response as it generates, reducing perceived latency
- Better extraction unit tests: A dedicated unit test suite for `extract_full_name()`, `extract_secondary_factor()`, etc. with hundreds of edge-case inputs
- Metrics instrumentation: Add Prometheus/OpenTelemetry counters for extraction hit rate, verification attempt distribution, card failure types, and session completion rate
- Confirmation flow improvements: Allow the user to change only the amount or only the card on cancel, not both

Medium-Term (1–2 months)

- Model upgrade path: Abstract the LLM backend so the same agent can run on llama3.1:8b locally or claude-haiku-4-5/gpt-4o-mini in production without code changes
- Redis-backed sessions: Replace the in-memory dict with Redis for horizontal scaling and session persistence
- Fuzzy name matching with confidence scoring: Exact match passes immediately; high Jaro-Winkler similarity (>0.95) prompts 'Did you mean X?' confirmation
- Multilingual support: Add language detection and route to multilingual models or prompts for Hindi and other Indian languages

Long-Term

- Active learning from failed verifications: Aggregate anonymized patterns to improve extraction models over time
- A/B testing framework: Run multiple response generation strategies in parallel and use conversion/satisfaction metrics to select winners
- Voice interface: The agent's state machine and extraction layer map cleanly to a voice IVR, the LLM response generator could be paired with TTS for a phone channel. Though this will need vigorous testing to prevent bad actors.